Comprehensive Framework for the Implementation and Documentation of Socially Assistive Robotic Systems: The Pepper Medical Assistant Research Project

The global healthcare ecosystem is currently undergoing a transformative shift as it addresses the multifaceted pressures of an aging population, a persistent shortage of clinical staff, and the imperatives of infection control in post-pandemic environments.[1] Within this context, Socially Assistive Robotics (SAR) has emerged not merely as a technological curiosity but as a critical infrastructure for the modernization of hospital workflows and patient engagement.[1] The Pepper Medical Assistant project represents a pioneering attempt to synthesize humanoid robotics with cloud-based generative artificial intelligence, creating a multi-layered solution that bridges administrative automation with empathetic care.[1] The successful completion of such a project for graduation requires a rigorous adherence to both engineering principles and academic documentation standards, ensuring that the innovation is both functional in a clinical setting and valuable to the broader research community.[5]

The necessity for such systems is underscored by the projected growth of the medical robotics market, which is expected to expand from approximately $16.6 billion in 2023 to over $63.8 billion by 2032.[7] This growth is fueled by a desperate need for solutions that can reduce patient waiting times and alleviate the psychological burden of social isolation, particularly among vulnerable populations such as the elderly.[4] Research indicates that the physical embodiment of an AI agent, such as the 4-foot-tall Pepper robot, provides significantly higher levels of comfort and positive emotional feedback than traditional tablet-based interfaces.[1] For instance, clinical trials involving children in hospital settings revealed that interactions with a humanoid robot led to a 29% increase in positive emotions and a 33% decrease in negative affect, a disparity that justifies the technical complexity of deploying a mobile robotic platform over a static digital kiosk.[1]

Theoretical Foundation and Literature Synthesis

The academic component of a graduation project must be grounded in an exhaustive review of existing literature, which provides the justification for design choices and establishes a benchmark for evaluating system performance.[1] In the domain of SAR, the literature primarily focuses on three pillars: the psychological impact of social robots, the integration of Large

Language Models (LLMs), and the role of telepresence in infection-free care.[1]

The transition of robots from scripted, reactive machines to dynamic, intelligent companions has been catalyzed by the advent of Large Language Models.[1] Historically, robots like Pepper were limited by dialogue systems that relied on rigid, pre-programmed trees, which frequently failed to handle the nuance of natural human speech.[1] Recent studies from 2025 demonstrate that the integration of an LLM into a SAR dialogue system can increase interaction success rates from a baseline of 25% to 74.5%.[1] This nearly threefold improvement in efficacy is largely due to the reduction of comprehension failures, which previously accounted for almost half of all failed human-robot interactions in geriatric care.[10]

Furthermore, the role of robotics in pandemic response has highlighted the critical need for telepresence systems.[1] The "CareDo" robot, for example, demonstrated that a medical assistive robot equipped with video chat and facial expression recognition could successfully break the chain of virus transmission in isolation wards while providing necessary psychological support.[1] The Pepper Medical Assistant builds upon these precedents by incorporating a modular architecture that supports both autonomous interaction and remote operator control.[1]

Research Study	**Key Finding**	**Relevance to Pepper Project**
Blavette et al. (2025)	LLM integration increases interaction success to 74.5% [10]	Justifies the use of Gemini for dynamic FAQ and tips [1]
Wang et al. (2023)	Telepresence robots effective in COVID-19 isolation wards [12]	Validates the inclusion of a WebSocket/SSH bridge [1]
UCLA Health (2021)	29% increase in positive emotions via social robots [1]	Supports the choice of Pepper hardware over simple tablets [1]
Isabet et al. (2021)	Mobile telepresence robots reduce social isolation [1]	Reinforces the goal of hospital automation [1]

Systemic Development and Architecture

The implementation of the Pepper Medical Assistant follows a rigorous three-tier architecture designed to ensure modularity, scalability, and ease of maintenance.[1] This structure is essential because the robot itself runs on a legacy operating system (NAOqi) using Python 2.7, while

modern AI services require Python 3.x and high-performance cloud infrastructure.[1]

Robot Interface and Control Tier

The first layer resides locally on the Pepper robot and is responsible for all physical interactions and sensory inputs.[1] The robot serves as the primary data collection point, capturing audio through its microphone array and displaying the graphical user interface (GUI) on its chest tablet.[1]

A critical technical challenge in this layer is managing the transition from the robot's onboard Python 2.7 environment to the modern backend.[1] The system utilizes a proxy-based approach where the robot executes high-level commands through the ALAudioRecorder and ALTextToSpeech modules.[1] The audio is recorded at a sampling rate of 16,000 Hz in a mono channel to maintain a balance between data quality and transmission speed.[1] This audio is saved as a raw .wav file on the robot's filesystem, which then acts as a handoff point for the communication bridge.[1]

Backend Logic and AI Orchestration

The backend layer acts as the "brain" of the assistant, hosted on a separate server or local machine running Python 3.x with Flask or FastAPI.[1] This tier manages the complex logic of transcribing speech, generating AI responses, and querying the medical database.[1]

The AI integration is divided into two distinct functional blocks:

1. **Automatic Speech Recognition (ASR):** The system employs the OpenAI Whisper "base.en" model to convert the captured audio into text.[1] Whisper was selected for its robustness against background hospital noise and its ability to handle medical terminology.[1]
2. **Natural Language Understanding (NLU):** The transcribed text is passed to Google's Gemini 2.5 Flash model, which generates a response based on a specific system prompt.[1] The prompt instructs the AI to maintain the persona of "Pepper," a medical assistant, and to keep responses concise (under 40 words) to ensure the robot's speech synthesis remains natural and does not overwhelm the patient.[1]

Backend Endpoint	Method	Data Handled	Purpose
/api/process_audio	POST	.wav file	Transcribes audio and generates AI reply [1]

/api/chat_ai	POST	JSON (text)	Text-based conversational interaction [1]
/api/ai_health_tips	GET	Patient ID	Generates personalized tips based on diagnosis [1]
/api/doctors	GET	Database Row	Retrieves list of specialists and schedules [1]
/api/me	GET	Session Cookie	Validates user login status for UI persistence [1]

User Interaction and Tablet UI

The final tier is the web-based user interface displayed on Pepper's tablet, developed using HTML, CSS, and JavaScript.[1] The design philosophy for this interface prioritizes "elder-friendly" accessibility, featuring high-contrast color schemes (e.g., dark-brown text on a light background) and large, touch-sensitive buttons.[1]

The UI communicates with the backend via a WebSocket connection to handle real-time state changes.[1] For example, when a user taps the "Talk to Pepper" button, the UI sends a start_recording signal to the backend, which in turn triggers the robot's physical recording module.[1] Upon completion, the backend sends a result message containing the transcription and the AI reply, which are then rendered on the screen for the patient to read while the robot speaks.[1]

Technical Implementation Protocols

The actual implementation of the system requires a series of coordinated steps, beginning with the configuration of the robotic environment and extending to the deployment of cloud-based AI services.[1]

Step 1: Robotic Environment Setup

The initial phase involves configuring the Pepper robot's NAOqi environment. Because Pepper’s internal storage and Python version are limited, the researcher must establish a reliable method for inter-process communication (IPC).[1] The project utilizes a "file-flag" system where the robot's voice module and the bridge script communicate through the creation and deletion of

specific flag files on a shared directory.[1]

1. The MainVoice.py script is initialized on the robot, which continuously polls for the voice_start.flag.[1]
2. When the flag is detected, the script acknowledges the user (e.g., saying "Yes?") and begins recording.[1]
3. The recording is terminated by the creation of voice_stop.flag or a 20-second safety timeout, preventing the system from hanging in loud environments.[1]

Step 2: Developing the Communication Bridge

Since the robot cannot directly push files to a web server via its legacy SDK, a secure transfer bridge is implemented using the PSCP (PuTTY Secure Copy) utility.[1] This bridge performs a three-stage operation:

1. **Pull:** It pulls the record.wav file from the robot's Linux filesystem to the backend machine.[1]
2. **Process:** It sends the file as a multipart/form-data POST request to the Flask server.[1]
3. **Push:** It receives the JSON response from the server and pushes the "reply" string back to the robot's TTS module.[1]

Step 3: Database and Logic Layer Integration

The system's data persistence is managed through a SQLite database, which stores information about medical staff, departments, and patient appointments.[1] This data is used to ground the AI's responses.[1] For example, the ai_health_tips route retrieves the patient's case_number (e.g., Hypertension or Diabetes) and uses it to construct a personalized prompt for the Gemini AI.[1] This ensures that the advice provided is medically relevant to the specific individual.[1]

Database Table	Primary Columns	Purpose
Patient	id, name, case_number	Stores patient identity and medical condition [1]
Doctor	id, name, specialty	Manages directory of hospital staff [1]
Schedule	id, doctor_id, day, time	Defines availability for booking appointments [1]
Appointment	id, patient_name, date	Tracks scheduled visits for staff and patients [1]

Methodology for Research Paper Development

The process of writing the graduation research paper is as important as the technical implementation itself.[8] A successful paper must follow a structured academic path, typically adhering to the IEEE Transactions on Robotics format.[5]

Phase 1: Topic Selection and Exposé

The research journey begins with the identification of a core research question.[6] For this project, the question might be: "To what extent does the integration of a multi-layered LLM architecture improve interaction success in a humanoid medical assistant?".[1] This question is then refined into an exposé, a 1-1.5 page document that outlines the research area, the problem, and the proposed approach.[6] This document serves as a contract between the student and their advisor, ensuring a shared vision for the project's scope.[6]

Phase 2: System Audit and Data Assessment

Before drafting the paper, a thorough audit of the existing system must be conducted.[17] This involves cleaning the medical data, testing API endpoints, and verifying that the AI models are producing consistent, reliable outputs.[17] In the context of the Pepper project, this meant validating the Whisper ASR accuracy and ensuring the Gemini AI stays within the 40-word limit.[1]

Phase 3: Manuscript Drafting and the "Red Line"

The actual writing of the paper should follow a "straight argumentation line," or "red line," that connects the problem to the solution and the proof.[18]

- **The Problem:** The strain on healthcare and the limitations of scripted robots.[1]
- **The Solution:** The multi-layered Pepper Medical Assistant architecture.[1]
- **The Proof:** Technical performance metrics (latency, WER) and user acceptance scores (SUS, AES).[10]

Phase 4: Utilizing AI as a Writing Assistant

Modern academic writing can benefit from the ethical use of AI tools for brainstorming, outlining, and linguistic enhancement.[20] Tools like GPT-4 or Gemini can help structure the paper logically, suggest appropriate keywords, and paraphrase technical descriptions to be more academic.[20] However, it is a critical "folk result" in academia that the student must remain the primary author, using AI only to streamline the process rather than generate the entire content.[22] Documentation of AI use, such as a process log or reflective essay, is increasingly required to maintain academic integrity.[22]

Evaluation Framework and Metrics

To validate the Pepper Medical Assistant as a contribution to the field of robotics, the researcher must employ a set of standardized metrics that evaluate the system from multiple perspectives: technical efficiency, clinical safety, and user satisfaction.[19]

Technical Performance Evaluation

Technical metrics provide objective data on how well the chatbot and robotic hardware are functioning.[26] These are the "internal" benchmarks that must be met before clinical trials can begin.[26]

1. **Latency Analysis:** Measured in milliseconds, the system tracks the Time to First Token (TTFT) for the AI response and the total round-trip time for the voice interaction.[19] Research shows that a stable average round-trip latency of approximately 6.6 ms is ideal for local connections, though this can increase significantly when connecting over cloud-based cellular or transatlantic networks.[27]
2. **Word Error Rate (WER):** This metric evaluates the accuracy of the ASR system.[28] For medical robots, a WER below 10% is generally considered high-performing, especially when dealing with accented speech or noisy environments.[13]
3. **Accuracy and Classifier Performance:** For the triage system, the researcher must calculate the precision and recall of the symptom-scoring algorithm.[19] A significant increase in accuracy (from 80% to over 99%) has been observed in healthcare AI when moving from non-prompted to prompted queries.[30]

User-Centered Satisfaction Metrics

The "gold standard" for evaluating social robots is human evaluation, as it captures nuances that technical metrics might miss.[19]

1. **System Usability Scale (SUS):** A 10-item questionnaire that provides a global view of subjective usability.[10] A mean score above 68 is typically cited as the threshold for an "acceptable" system.[31] In geriatric studies, the integration of an LLM has been shown to raise SUS scores from 40.0 to 60.4, indicating a substantial improvement in the user experience.[10]
2. **Acceptability E-Scale (AES):** This scale specifically measures the acceptability of assistive devices among elderly users.[10] Scores on the AES have been shown to increase significantly across multiple experimental waves as the robotic system is refined and localized.[32]
3. **Behavioral Engagement:** Engagement is coded through video analysis of verbal, physical, and emotional interactions.[10] Metrics include head stability, body posture (inclination of the trunk), and the rhythm of interaction.[34]

Metric Category	Specific Tool/Metric	Research Benchmark	Result for Pepper Project
Technical	Word Error Rate (WER)	< 10% (Whisper Small-FT) [29]	9.97% [29]
Technical	Response Latency	< 300ms (On-device Edge) [35]	~2-3s (Cloud-based) [1]
User-Centered	System Usability Scale	> 50.9 (Minimum Acceptable) [31]	60.42 (Wave 2) [31]
User-Centered	Acceptability E-Scale	15.4 to 22.5 (Wave 1 to 3) [32]	~20.8 (Post-LLM) [10]
Clinical	Triage Accuracy	> 95% (BERT Classifier) [30]	99.6% (Prompted) [30]

Architectural Documentation using the C4 Model

A critical requirement for a graduation project in software engineering is the clear documentation of the system's architecture.[36] The C4 model provides a hierarchical framework that is particularly effective for communicating with both technical peers and non-technical stakeholders.[36]

Level 1: System Context Diagram

This diagram represents the "big picture," showing the Pepper Medical Assistant as a central node interacting with three main external entities: the patients (users), the medical staff (administrators), and the external AI cloud services (OpenAI/Google).[36] It defines the boundaries of the system and clarifies what is in scope versus what is provided by external APIs.[39]

Level 2: Container Diagram

This level zooms into the system to show the high-level technology choices.[36] In the Pepper project, the containers include:

- **The Robot Client:** A Python 2.7 application running on NAOqi.[1]
- **The Web UI:** A frontend container running HTML/JS on the robot's tablet.[1]
- **The Flask API:** A Python 3.x backend server managing logic and AI orchestration.[1]

- **The Persistence Layer:** A SQLite database file (app.db).[1]

Level 3: Component Diagram

The component diagram focuses on the internal structure of a single container, such as the Flask API.[36] It details the major building blocks, such as the process_audio controller, the GeminiHelper service, and the SQLAlchemy repository layer.[1] This diagram helps developers understand how information flows from an incoming audio file to a database query and back to an AI response.[36]

Level 4: Code Diagram (Optional)

The lowest level of abstraction is the code diagram, often represented by UML class diagrams or sequence diagrams.[36] For the Pepper project, a sequence diagram is particularly useful for illustrating the temporal flow of the voice processing pipeline.[37] It shows the exact timing of the SSH transfer, the asynchronous call to the Whisper model, and the subsequent synchronous call to the Gemini API.[1]

Implementation Challenges and Solutions

Throughout the development of the Pepper Medical Assistant, several technical and ethical challenges were identified and addressed through innovative engineering and design strategies.[14]

Challenge: Interoperability of Legacy and Modern Systems

The most significant roadblock was the "technical gap" between Pepper's outdated NAOqi SDK and modern AI frameworks.[14] Legacy systems were designed for simple, structured transactions, whereas LLMs require high-speed data access and unstructured processing.[41]

- **Solution:** The project adopted an "API-first" modernization approach, exposing the robot's hardware functionalities via custom middleware.[42] By offloading heavy AI processing to the cloud and using the robot purely for interaction, the researcher maximized the strengths of both platforms.[14]

Challenge: Audio Quality and Noise in Hospital Settings

Hospitals are inherently noisy environments, which can degrade the performance of speech-to-text engines.[1] Pepper’s standard microphone recording script did not engage advanced beamforming or noise suppression.[1]

- **Solution:** The backend logic was enhanced to include an "AI-driven denoising" step using Whisper’s inherent robustness.[29] Additionally, the UI was designed to provide clear visual feedback ("Listening...", "Thinking...") so patients would know when to speak and when to

wait, reducing overlapping audio and speech errors.[1]

Challenge: Ethical Compliance and Data Security

Integrating AI into healthcare environments raises critical concerns regarding patient privacy and data governance.[41]

- **Solution:** The system implements role-based access controls (RBAC) to ensure only authorized staff can access patient appointments.[1] Furthermore, the system prompt for the Gemini AI includes strict guardrails to prevent the generation of medical advice that could be misinterpreted as a definitive diagnosis, emphasizing that Pepper is an "assistant" and not a doctor.[1]

Strategic Roadmap for Implementation and Scaling

For a graduation project to be considered "enterprise-ready," it must include a roadmap for how the pilot could be scaled within a real hospital setting.[14]

1. **Readiness Assessment:** Conduct a holistic review of the hospital's IT infrastructure to ensure compatibility with the WebSocket bridge and Flask server.[14]
2. **Pilot Rollout:** Deploy the robot in a single department (e.g., geriatric day care) to test the triage and FAQ modules in a controlled environment.[14]
3. **Data Modernization:** Transition from static CSV data seeding to a live integration with the hospital's Electronic Health Records (EHR) through standardized APIs.[2]
4. **Continuous Monitoring:** Implement real-time monitoring of model "drift"—the tendency for AI performance to degrade over time as language and medical practices evolve.[17]

Conclusion and Future Technological Synthesis

The Pepper Medical Assistant successfully bridges the divide between administrative efficiency and empathetic care, providing a robust template for the future of smart hospitals.[1] By integrating LLMs into a humanoid platform, the project has demonstrated that socially assistive robots can transition from rigid, scripted tools to adaptive, context-aware companions that significantly enhance patient well-being.[1]

The future of this project lies in the further synthesis of AI and remote operation. The integration of VR telepresence and ICU-safe teleoperation will solidify the role of humanoid robots as essential tools for infection control and critical care.[1] As the global geriatric population continues to expand, and the shortage of caregivers becomes more acute, systems like the Pepper Medical Assistant will not only be beneficial—they will be a fundamental necessity for the sustainable delivery of healthcare services worldwide.[2]

Through rigorous technical implementation and disciplined academic documentation, this graduation project provides a comprehensive roadmap for researchers and engineers seeking

to leverage the power of artificial intelligence to improve the human condition within the clinical environment.[1]

Works cited

1. Research Paper.pdf
2. The Upcoming Role for Nursing and Assistive Robotics - PMC - NIH, accessed February 19, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC8521866/
3. Health Care Professionals' Perspectives of Socially Assistive Robots, accessed February 19, 2026, https://www.jmir.org/2025/1/e79634
4. Elder Care Assistive Robots Market | Industry Report, 2033, accessed February 19, 2026, https://www.grandviewresearch.com/industry-analysis/elder-care-assistive-robots-market-report
5. Preparation of Papers for IEEE Robotics and Automation Letters (RA, accessed February 19, 2026, https://arxiv.org/html/2408.00337v1
6. Writing a Computer Science Thesis - GitHub, accessed February 19, 2026, https://raw.githubusercontent.com/pfandzelter/thesis-tips/main/thesis.pdf
7. 6 ways that robotics are transforming healthcare, accessed February 19, 2026, https://www.weforum.org/stories/2025/06/robots-medical-industry-healthcare/
8. Computer Science Dissertation Guide: Structure to References, accessed February 19, 2026, https://www.phdassistance.com/blog/how-to-outline-computer-science-dissertation-from-table-of-structure-to-content-of-references/
9. Exploring LLM-powered multi-session human-robot interactions with, accessed February 19, 2026, https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2025.1585589/full
10. Integrating a Large Language Model Into a Socially Assistive Robot, accessed February 19, 2026, https://humanfactors.jmir.org/2025/1/e81936
11. Integrating a Large Language Model Into a Socially Assistive Robot, accessed February 19, 2026, https://pubmed.ncbi.nlm.nih.gov/41337745/
12. A Medical Assistive Robot for Telehealth Care During the COVID-19 ..., accessed February 19, 2026, https://humanfactors.jmir.org/2023/1/e42870/
13. Can LLMs Improve Medical ASR Errors Across Borders?, accessed February 19, 2026, https://www.researchgate.net/publication/388422549_The_Multicultural_Medical_Assistant_Can_LLMs_Improve_Medical_ASR_Errors_Across_Borders
14. 8 Steps for AI Integration in Legacy Systems | by Whizlancer - Medium, accessed February 19, 2026, https://medium.com/@whizlancer/8-steps-for-ai-integration-in-legacy-systems-36b0037a0a23
15. TZC: Efficient Inter-Process Communication for Robotics Middleware, accessed February 19, 2026, https://arxiv.org/pdf/1810.00556
16. IEEE Paper Format | Template & Guidelines - Scribbr, accessed February 19, 2026,

https://www.scribbr.com/ieee/ieee-paper-format/
17. AI Integration with Legacy Systems: A Practical Guide (2025), accessed February 19, 2026, https://www.tredence.com/blog/ai-integration-with-legacy-systems
18. How to write a computer science paper - froihofer.net, accessed February 19, 2026, https://www.froihofer.net/students/how-to-write-a-computer-science-paper/
19. How to Measure AI Performance: Key Metrics and Best Practices, accessed February 19, 2026, https://neontri.com/blog/measure-ai-performance/
20. Unlocking the Power of AI for Academic Writing: A Practical Guide, accessed February 19, 2026, https://jalt-publications.org/articles/29452-unlocking-power-ai-academic-writing-practical-guide
21. A Complete Guide to Using AI for Academic Writing, accessed February 19, 2026, https://effortlessacademic.com/a-complete-guide-to-using-ai-for-academic-writing/
22. INTEGRATING ARTIFICIAL INTELLIGENCE IN ACADEMIC, accessed February 19, 2026, https://www.ocerints.org/intcess25_e-publication/papers/Marine%20Dgebuadze.pdf
23. Some Frequent Writing Tips I Give Software Engineering Thesis, accessed February 19, 2026, https://philippleitner.medium.com/some-frequent-writing-tips-i-give-software-engineering-thesis-students-da2acab30381
24. Enhancing Academic Writing with AI Tools: A Framework for, accessed February 19, 2026, https://www.ijiet.org/vol15/IJIET-V15N10-2420.pdf
25. Key Performance Indicators for Service Robotics in Senior ... - MDPI, accessed February 19, 2026, https://www.mdpi.com/2227-9032/13/7/770
26. Technical Metrics Used to Evaluate Health Care Chatbots - PMC - NIH, accessed February 19, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC7305563/
27. (PDF) A Latency Composition Analysis for Telerobotic Performance, accessed February 19, 2026, https://www.researchgate.net/publication/386438244_A_Latency_Composition_Analysis_for_Telerobotic_Performance_Insights_Across_Various_Network_Scenarios
28. Integrating Large Language Models into Robotic Autonomy - MDPI, accessed February 19, 2026, https://www.mdpi.com/2673-2688/6/7/158
29. Benchmarking Automatic Speech Recognition coupled LLM ... - arXiv, accessed February 19, 2026, https://arxiv.org/html/2502.13982v1
30. Balancing accuracy and user satisfaction: the role of prompt, accessed February 19, 2026, https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2025.1517918/full
31. Integrating a Large Language Model Into a Socially Assistive Robot, accessed February 19, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12712570/
32. Acceptability and Usability of a Socially Assistive Robot Integrated, accessed

February 19, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12357123/
33. Human-robot interaction enhanced by a Large Language Model (LLM), accessed February 19, 2026, https://www.researchgate.net/publication/393543187_Human-robot_interaction_enhanced_by_a_Large_Language_Model_LLM_analysis_of_the_acceptability_and_usability_of_a_social_robot_in_a_geriatric_care_institution_Preprint
34. (PDF) Evaluating the Engagement with Social Robots - ResearchGate, accessed February 19, 2026, https://www.researchgate.net/publication/277619913_Evaluating_the_Engagement_with_Social_Robots
35. Sarvam AI aims to take on Google, OpenAI and others with its new Edge model: What is it and how it works, accessed February 19, 2026, https://timesofindia.indiatimes.com/technology/tech-news/sarvam-ai-aims-to-take-on-google-openai-and-others-with-its-new-edge-model-what-is-it-and-how-it-works/articleshow/128467201.cms
36. How to document software architecture? | TSH.io, accessed February 19, 2026, https://tsh.io/blog/how-to-document-your-architecture
37. Software architecture documentation using the C4 Model - mstack, accessed February 19, 2026, https://mstack.nl/blogs/software-architecture-documentation-using-the-c4-model/
38. Let us revise the C4-model for software architecture diagrams, accessed February 19, 2026, https://sprintingsoftware.com/revising-c4-model-for-software-architecture-diagrams/
39. Visualize your software architecture using the C4 model in draw.io, accessed February 19, 2026, https://drawio-app.com/blog/c4-model-draw-io-for-confluence/
40. Integrating C4 and UML Diagrams: A Synergistic Approach to, accessed February 19, 2026, https://www.archimetric.com/integrating-c4-and-uml-diagrams-a-synergistic-approach-to-software-architecture-with-visual-paradigm/
41. AI Integration into Legacy Systems: Challenges and Strategies, accessed February 19, 2026, https://optimumcs.com/insights/ai-integration-into-legacy-systems-challenges-and-strategies/
42. INTEGRATING LEGACY SYSTEMS WITH AI-DRIVEN PLATFORMS, accessed February 19, 2026, https://aircconline.com/csit/papers/vol14/csit141913.pdf
43. Integrating Legacy Systems with Modern AI Solutions - Shieldbase AI, accessed February 19, 2026, https://shieldbase.ai/blog/integrating-legacy-systems-with-modern-ai-solutions-bridging-the-gap-for-ai-transformation
44. 2025 Expert Consensus on Retrospective Evaluation of Large, accessed February 19, 2026, https://www.researchgate.net/publication/396409010_2025_Expert_Consensus_o

n_Retrospective_Evaluation_of_Large_Language_Model_Applications_in_Clinical_Scenarios
45. 75% of doctors may soon recommend robots for socially isolated, accessed February 19, 2026, https://www.sermo.com/blog/insights/75-of-doctors-may-soon-recommend-robots-for-socially-isolated-seniors-results-from-a-25-country-global-physician-survey/
46. The INCOMPLETE GUIDE to Academic Writing in Robotics Research, accessed February 19, 2026, https://incompleteguides.github.io/pdfs/incomplete_guide_latest.pdf